

Boot Spring Boot

스프링 부트 이용설명서

Written by honeymon

Boot Spring Boot

부트 스프링 부트

Jake Jiheon Kim

Version 0.0.2, 2016-10-10

Table of Contents

저자 소개	2
들어가며.....	4
책에서 다루는 내용	4
책을 따라하기 위해 필요항목.....	5
대상독자.....	5
관례(Convention)	5
예제	6
1. 시작하기	8
1.1. 스프링 부트 소개.....	8
1.2. 시스템 요구사항	9
1.3. 스프링 부트 설치.....	9
1.3.1. 설치방법 소개	9
1.4. 스프링 부트 애플리케이션 개발	9
1.4.1. 프로젝트 생성	10
1.4.2. 의존성 추가	10
1.4.3. 코드 작성하기	10
1.4.4. 실행하기	10
1.4.5. 실행가능한 jar 생성	10
1.5. 이어질 이야기.....	10
2. 첫 스프링 부트 애플리케이션 살펴보기.....	11
2.1. 빌드 시스템.....	11
2.1.1. 의존성 관리	11
2.1.2. 메이븐	11
2.1.3. 그레이들	11
2.1.4. 스타터	11
2.2. 코드 구조	11
2.2.1. "기본" 패키지 이용	11
2.2.2. 애플리케이션 메인 클래스 위치	11
2.3. 구성(Configuration) 클래스.....	11
2.3.1. 추가적인 구성클래스 불러오기	11
2.3.2. XML 구성 불러오기	11
2.4. 자동구성(Auto-configuration)	11
2.4.1. 자동구성 대체하기.....	11
2.4.2. 특정 자동구성 비활성화하기	11
2.5. 스프링 빈과 의존성 주입	12
2.6. @SpringBootApplication 애노테이션 이용	12
2.7. 애플리케이션 실행하기	12
2.7.1. IDE 에서 실행하기.....	12
2.7.2. 패키징된 애플리케이션 실행하기	12
2.7.3. 메이븐 플러그인으로 실행하기	12
2.7.4. 그레이들 플러그인으로 실행하기	12
2.7.5. 핫 스와핑	12
2.8. 스프링 부트 개발자도구	12
2.8.1. 기본 속성들	12
2.8.2. 자동 재시작	12

2.8.3. 전역 설정	12
2.8.4. 원격 애플리케이션	12
2.9. 출시를 위한 애플리케이션 패키징	12
2.10. 이어질 이야기	12
3. 스프링 부트 기능	13
3.1. 스프링애플리케이션(SpringApplication)	13
3.1.1. 배너 재정의	13
3.1.2. SpringApplication 재정의	13
3.1.3. 유연한(Fuent) 빌더 API	13
3.1.4. 애플리케이션 이벤트 및 리스너	13
3.1.5. 웹 환경	13
3.1.6. 애플리케이션 실행 인자에 접근	13
3.1.7. ApplicationRunner 혹은 CommandLineRunner 사용하기	13
3.1.8. 애플리케이션 종료	13
3.1.9. 관리자 기능	13
3.2. 외부 구성	13
3.2.1. 랜덤값 구성	13
3.2.2. 명령줄 속성 접근	13
3.2.3. 애플리케이션 속성 파일	13
3.2.4. 프로파일 정의 속성	13
3.2.5. 속성 내 치환자(placeholder)	13
3.2.6. properties 를 대신하여 YAML 사용하기	13
3.2.7. 보장형 구성 속성	14
3.3. 프로파일	14
3.3.1. 활성화할 프로파일 추가하기	14
3.3.2. 프로파일을 프로그래밍적으로 설정	14
3.3.3. 프로파일 정의 구성 파일	14
3.4. 로깅	14
3.4.1. 로그 형식	14
3.4.2. 콘솔 출력	14
3.4.3. 파일 출력	14
3.4.4. 로그 레벨	14
3.4.5. 로그 구성 재정의	14
3.4.6. 로그백 확장	14
3.5. 웹애플리케이션 개발	14
3.5.1. 'Spring Web MVC framework'	15
3.5.2. 내장 서블릿 컨테이너 지원	15
3.6. 보안	15
3.6.1. OAuth2	15
3.6.2. 사용자 정보 속 토큰 유형	15
3.6.3. 사용자정보 RestTemplate 재정의	15
3.6.4. 액추에이터 보안(Actuator Security)	15
3.7. SQL 데이터베이스 이용	15
3.7.1. DataSource 구성	15
3.7.2. JdbcTemplate 사용	15
3.7.3. JPA 그리고 '스프링 Data'	15
3.7.4. H2 사용	15

3.8. NoSQL 기술 이용.....	16
3.8.1. 레디스(Redis).....	16
3.8.2. 몽고DB(MongoDB).....	16
3.8.3. 솔(Solr).....	16
3.8.4. 엘라스틱서치(Elasticsearch).....	16
3.9. 캐시.....	16
3.9.1. 지원하는 캐시 제공자(Provider).....	16
3.10. REST 서비스 호출.....	17
3.10.1. RestTemplate 재정의.....	17
3.11. 스프링 세션.....	17
3.12. 테스트.....	17
3.12.1. 테스트 관련 의존성.....	17
3.12.2. 스프링 애플리케이션 테스트.....	17
3.12.3. 스프링 부트 애플리케이션 테스트.....	17
3.12.4. 테스트 유틸리티.....	17
3.13. 웹소켓(WebSocket).....	18
3.14. 웹 서비스(Web Service).....	18
3.15. 자동구성 직접 생성하기.....	18
3.15.1. 자동구성 빈 이해하기.....	18
3.15.2. 자동구성 대상자 위치.....	18
3.15.3. 조건 애너테이션.....	18
3.15.4. 스타터 직접 생성하기.....	18
3.16. 이어질 이야기.....	18
4. 스프링 부트 액츄에이터: 출시준비기능.....	19
4.1. 출시준비 기능 활성화.....	19
4.1.1. 종단점(Endpoint).....	19
4.1.2. 종단점 재정의.....	19
4.1.3. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어.....	19
4.1.4. CORS 지원.....	19
4.1.5. 재정의 종단점 추가하기.....	19
4.1.6. 건강정보.....	19
4.1.7. HealthIndicator 를 사용한 보안.....	19
4.1.8. 애플리케이션 정보.....	19
4.2. HTTP 로 모니터링하고 관리하기.....	19
4.2.1. 종단점 에 대한 노출강도 보안관리.....	19
4.2.2. 관리 종단점 경로 재정의.....	19
4.2.3. 관리 서버 포트 재정의.....	19
4.2.4. 관리를 위한 SSL 재정의.....	19
4.2.5. 관리 서버 주소 재정의.....	20
4.2.6. HTTP 종단점 비활성화.....	20
4.2.7. HTTP 건강 종단점 접근 제한.....	20
4.3. 원격셀을 통해 모니터링하고 관리하기.....	20
4.3.1. 원격셀로 접속.....	20
4.3.2. 원격셀 확장.....	20
4.3.3. 원격셀 플러그인.....	20
4.4. 측량(Metrics).....	20
4.4.1. 시스템 측량.....	20
4.4.2. 데이터소스 측량.....	20

4.4.3. 캐시 측량	20
4.4.4. 토크 세션 측량	20
4.4.5. 측량 기록	20
4.4.6. 공개 측량 추가	20
4.5. 감시	20
4.6. 추적	20
4.7. 프로세스 모니터링	20
4.7.1. 확장 구성	20
4.7.2. 프로그래밍적인 처리	20
4.8. 이어질 이야기	20
5. 스프링 부트 애플리케이션 배포	21
5.1. 클라우드 배포	21
5.1.1. 아마존웹서비스(AWS, Amazon Web Service)	21
5.1.2. 구글앱엔진	21
5.1.3. 클라우드 파운드리(Cloud Foundry)	21
5.2. 스프링 부트 애플리케이션 설치	21
5.2.1. 유닉스/리눅스 서비스	21
5.3. 마이크로소프트 윈도우즈 서비스	21
5.4. 이어질 이야기	21
6. 빌드 도구 플러그인	22
6.1. 스프링 부트 메이븐 플러그인	22
6.1.1. 플러그인 포함	22
6.1.2. 실행가능한 jar 와 war 파일 패키징	22
6.2. 스프링 부트 그레이들 플러그인	22
6.2.1. 플러그인 포함	22
6.2.2. 그레이들 의존성 관리	22
6.2.3. 실행가능한 jar 와 war 파일 생성	22
6.2.4. 스프링 부트 플러그인 구성	22
6.2.5. 리패키징 구성	22
6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기	22
6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포	22
6.3. 다른 빌드 시스템 지원	22
6.3.1. 리패키징 아카이브	22
6.3.2. 내포된 라이브러리	22
6.3.3. 메인 클래스 찾기	22
6.3.4. 리패키지 구현 예제	22
6.4. 이어질 이야기	22
7. 스프링 부트 활용 안내	23
7.1. 스프링 부트 애플리케이션	23
7.1.1. 자동구성 문제해결	23
7.1.2. Environment 혹은 ApplicationContext 가 시작하기 전에 재정의하기	23
7.1.3. ApplicationContext 계층 조직(부모 혹은 루트 컨텍스트 추가)	23
7.1.4. 비-웹애플리케이션 생성	23
7.2. 속성 및 구성	23
7.2.1. 빌드 시 자동으로 속성 확장	23
7.2.2. SpringApplication 외부 구성	23
7.2.3. 애플리케이션 속성 외장파일 위치변경	23
7.2.4. 명령줄 인자 축약사용	23

7.2.5. 외부속성에 대한 YAML 사용	23
7.2.6. 스프링 프로파일 활성화	23
7.2.7. 환경 의존적인 구성 변경	23
7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견	23
7.3. 내장서블릿 컨테이너	23
7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기	23
7.3.2. HTTP 포트 변경	23
7.3.3. 할당되지 않은 HTTP 포트 무작위 사용	23
7.3.4. 실행시 HTTP 포트 발견	24
7.3.5. SSL 구성	24
7.3.6. 접근로그 구성	24
7.3.7. 프론트엔드 프록시 서버 이용	24
7.3.8. 톰캣 구성	24
7.3.9. 톰캣을 통한 다중 커넥터 활성화	24
7.3.10. 톰캣 7.x 혹은 8.0 이용	24
7.3.11. <code>@ServerEndpoint</code> 를 사용한 웹소켓 종단점 생성	24
7.3.12. HTTP 응답 압축 활성화	24
7.4. 스프링 MVC	24
7.4.1. JSON REST 서비스 작성	24
7.4.2. XML REST 서비스 작성	24
7.4.3. 잭슨(Jackson) 오브젝트매퍼(<code>ObjectMapper</code>) 재정의	24
7.4.4. <code>@ResponseBody</code> 렌더링 재정의	24
7.4.5. Multipart 파일 업로드 제어	24
7.4.6. 스프링 MVC <code>DispatcherServlet</code> 전환	24
7.4.7. 기본 MVC 구성 전환	24
7.4.8. <code>ViewResolver</code> 재정의	24
7.5. HTTP 클라이언트	24
7.5.1. <code>RestTemplate</code> 가 프록시를 사용하도록 구성	24
7.6. 로깅	24
7.6.1. Logback 구성	25
7.6.2. Log4j 구성	25
7.7. 데이터 접근	25
7.7.1. <code>DataSource</code> 구성	25
7.7.2. <code>DataSource</code> 복수 정의	25
7.7.3. 스프링 Data 리파지토리 이용	25
7.7.4. 스프링 구성으로부터 <code>@Entity</code> 정의 분리	25
7.7.5. JPA 속성 구성	25
7.7.6. <code>EntityManagerFactory</code> 재정의 이용	25
7.7.7. 복수 <code>EntityManager</code> 이용	25
7.7.8. 전통적인 <code>persistence.xml</code> 이용	25
7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용	25
7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출	25
7.8. 데이터베이스 초기화	25
7.8.1. JPA 를 이용한 데이터베이스 초기화	25
7.8.2. Hibernate 를 이용한 데이터베이스 초기화	25
7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화	25
7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화	25

7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용	25
7.9. 배치 애플리케이션	26
7.9.1. 구동시 스프링 배치 작업 실행	26
7.10. 액추에이터	26
7.10.1. 액추에이터 종단점 의 포트 혹은 주소 변경	26
7.10.2. whitelabel 오류 페이지 재정의	26
7.10.3. 액추에이터 과 Jersey	26
7.11. 보안	26
7.11.1. 스프링 부트 보안 구성 전환	26
7.11.2. AuthenticationManager 변경하고 사용자 계정 추가	26
7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화	26
7.12. 핫스와핑(Hot swapping)	26
7.12.1. 정적 콘텐츠 재적재	26
7.12.2. 컨테이너 재시작없이 템플릿 재적재	26
7.12.3. 빠른 애플리케이션 재시작	26
7.13. 빌드	26
7.13.1. 빌드 정보 생성	26
7.13.2. 깃 정보 생성	26
7.13.3. 의존성 버전 재정의	26
7.13.4. Java 6 에서 실행하기	26
7.14. 전통적 배포	26
7.14.1. 배포가능한 war 파일 생성	26
7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성	27
7.14.3. 기존 애플리케이션을 스프링 부트 로 변환	27
7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포	27
8. 정리	28
부록	29
Appendix A: 개발환경 설정	30
JDK설치	30
STS 설치	30
그레이들 설치	30
Appendix B: 개발 시 참고사항	31
스프링부트 레퍼런스 문서 참고	31
관련 자동구성 클래스 확인	31
debug=true 를 이용한 활성화된 자동설정과 비활성화된 자동설정 확인	31
실행가능한 jar 구조	31
공통 애플리케이션 속성들	31
스택오버플로에서 스프링부트 관련 질문 찾기	31
Appendix C: 참고문헌	32
Spring Framework	32
Spring Boot	32
Spring Security	32
Spring Test	32

저자 소개

안녕하세요, 김지현입니다.

인터넷에서는 허니몬(hoeymon)이라는 이름으로 활동하고 있습니다.

항상 자기소개를 할 때면 수줍습니다.

'일반인'을 자처하면서 '개발자'가 읽을 책을 써보겠다고 나서는 무모함을 가지고 있는 평범한 사람입니다. 스쿠버다이빙, 클라이밍, 라이딩, 수영 등의 취미생활을 즐기는 것 만큼 개발자로서 일해오는 것을 즐겁게 생각하고 있습니다. 생물학과 컴퓨터과학을 복수전공하면서 어느쪽으로 갈까 고민하다가 잠시 전산쪽으로 빠져 있었습니다. 그러다가 그쪽의 일이 들어지면서 진로를 고민하던 차에 사촌 덕분에 이 바닥에 뛰어들게 되었습니다. 년수로는 8년차가 되었네요. 빠르게 발전하고 변화해가는 IT기술 속에서 '자바' 그 중에서 스프링 을 기반으로 한 '웹 애플리케이션' 을 개발하게 된 건 어찌보면 당연하다 여겨질만큼 우리나라 시장에서 많은 수요가 있습니다. 대부분은 SI(System Integration) 라고 하는 특수한 환경이 비정상적으로 비대해지면서 생겨난 특이한 상황이죠. 저는 그런 상황 속에서 꽤 운이 좋은 편이었습니다. 5개월의 '정부지원 자바 전문가 과정'을 마치고 공공기관의 유지보수(SM, System Maintenance) 에 투입되어 1년 동안 생활하면서 '이건 나랑 안맞아' 라고 생각하고 회사에 배치변경을 요구했지만 묵살당하면서 그에 대한 반감을 가지면서였습니다. 때마침 '이일민'님께서 '토비의 스프링 3.0'을 출간하면서 한국스프링유저그룹(KSUG, Korea Spring User Group)에서 책임기 모임이 있었습니다. 그 책임기 모임을 시작으로 KSUG 와의 길고긴 인연이 지금까지 이어져서 현재는 KSUG 일꾼단에서 3년 넘게 활동을 하고 있습니다. 그 덕분에 이런저런 경험을 하면서 많은 분들을 만나면서 조금은 개발자 흉내를 낼 수 있게 된 듯 합니다.

제가 스프링 부트 와 인연을 맺게 된 것은, 2014년 5월 말에 잘 다니던 회사를 그만두고 '스쿠버 다이빙 강사' 가 되겠다며 동해바다로 갔다가 3개월 동안 생활하다가 돌아와 스터디에서 알던 형과 함께 솔루션 개발에 스프링 부트 를 적용했다.

다양한 운영체제에도(JVM 기반) 단독 실행이 가능한(Executable jar) 웹애플리케이션(Spring MVC)으로 개발해야한다.

는 요구사항에 제일 먼저 떠오른 것이 '스프링 부트' 였다. 그리고 거기에 네이버에서 오픈소스로 개발하던 요비(YOBI)를 깃과 함께 사용하기를 결정했다. 스프링 부트 로 개발을 하면서 제일 많이 보게되는 것은 역시 [참고문서\(refernece document\)](#) 였다. 스프링 은 정말 문서화가 잘 되어 있다. 오픈소스이고 거기에 문서화도 [아스키독\(Asciidoc\)](#) 으로 작성되어 있었기에 번역을 해보기로 결심했다. 이 번역을 하기에 앞서 에이콘에서 '[Git을 이용한 버전관리](#)' 라는 책을 번역했었기에 쉽게 봤다(... 부끄럽고 죄송하다).

'무식하면 용감하다'

라던가? [Spring Boot Reference Guide](#) 를 그대로 복사하여 번역할 계획을 세운다. 이제는 요령이 생겼지만, 저걸 처음 시작할 때는 'current' 와 'current-snap' 과 특정 버전의 문서를 각각 볼 수 있다는 것과, 스프링 부트 가 빠르게 버전업된다는 것을 살피지 못했다. 그리고 용감하게 번역을 시작한다.

• 스프링 부트 참고가이드

저장소에 들어오면 바로 볼 수 있도록 하겠다는 생각으로 [README.md](#) 에 작성을 시작했다. 정말 무식하게 했다. 그러다가 지쳐서 딴짓 하다가 다른 분들에게 도움을 받아 80% 정도의 번역을 끝으로 손을 뗐다(... 마무리를 지어볼까 하는 시점에는 1.3.0 이 나와버렸고 제대로 구성하지 않은 문서를 가지고는 변경사항들만 적용하기도 어려웠다). 이 무모했던 번역을 마무리 짓고 있던 내게 또 다른 번역 제안이 들어왔다. 그리고 5개월 동안 번역하고 손질하는데 많은 시간이 들고서 '[Spring MVC 4 익히기](#)' 라는 책이 나왔다. 이 책을 번역하고 있는 동안에 몇 곳에서 스프링 부트 관련 번역서 제안과 집필

제안이 있었다.

그리고 현재 이 책을 쓰고 있다. 두둑치…!

들어가며

스프링 부트 가 소개(2014.04.01)된 지 2년이 넘은 현재 시점에 들어서 스프링 부트 에 대한 책들이 나오기 시작했다. 각각의 책들은 각자의 방식으로 스프링 부트 를 소개하고 있으며, 이 책을 쓰고 있는 나 역시 나만의 방식으로 스프링 부트 를 여러분이 사용할 수 있도록 유혹하려고 한다. **스프링 부트 참고가이드** 를 번역했던 경험을 바탕으로 스프링 부트 로 개발하면서 정말 '스프링 부트 참고 문서'를 많이 봤다. 거기서 나아가 소스코드를 열어보면서 어떻게 구성되어 있고 동작할지를 살피기 시작했고 그런 이해를 바탕으로 필요한 경우 구성을 변경하거나 제공하는 클래스와 인터페이스를 사용하여 직접 구성하는 경험을 쌓게 되었다. 그런 경험을 전달하여 삽질(or 시행착오)을 줄이면서 빠르게 적응하길 바라는 마음으로 이 책을 쓴다.

이 책은 스프링 부트 참고문서를 기반으로 하여 그에 대응하는 설명과 '삽질경험' 을 기술한다.

책에서 다루는 내용

시작하기에서는, 스프링 부트 에 대해서 간단하게 소개하면서 스프링 부트 의 기본적인 개발흐름을 눈에 익혀본다. 스프링 부트는 그 이전에 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 개발경험을 개발자들에게 선사해 주었다. 이는 다른 언어가 가지고 있던 장점들을 자바 개발자들에게 부여하며 '빠른 프로토타이핑' 및 '개발/운영' 까지 이어지도록 해주는 근사한 경험을 선물한다. 거기에는 문맥(Context) 라고 하는 흐름이 구성(Configuration) 이라는 개념으로 변모해가는 역동적인 변화가 발생한다.

첫 스프링 부트 애플리케이션 살펴보기에서는 스프링 부트 를 본격적으로 들여다본다. 기능구현에 필요한 다른 라이브러리들을 손쉽게 추가할 수 있는 '의존성 관리', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '자동구성(Auto-Configuration)', 개발 생산성을 높여주고 확장성을 부여하는 '스프링 프레임워크'와의 조합, 내장컨테이너를 이용한 빠른 실행과 재가동 시간을 줄이는 '개발자 도구' 등 스프링 부트 를 이용하는데 알아둬야할 것들을 살펴본다.

스프링 부트 기능에서는 스프링 부트 를 자유자재로 응용할 수 있는 구간이 되는 스프링 부트의 기능들을 살펴본다. 스프링 부트 의 진입점(Entry point)가 되는 **SpringApplication** 을 시작으로 해서 배포를 위해 구성된 배포파일을 다시 빌드하지 않아도 실행환경에 따라 설정값을 변경할 수 있으며, 프로파일을 통해서 '로컬', '단위테스트', '개발', '운영' 등의 독립적인 실행환경별로 구성하고, 다양한 생태계와 연동하고, 필요에 따라 추가적인 구성을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 **스프링 부트**의 매력에 흠뻑 취하게 될 것이다.

스프링 부트 액추에이터: 출시준비기능에서는 '빠른 개발' 이후에 고민하게 되는 '운영' 의 묘미를 살려줄 수 있는 다양한 기능을 살펴본다. 개발하여 배포하고 운영되고 있는 스프링 부트 를 모니터링하고 그 정보를 수집하고 경우에 따라 쉽게 제어할 수 있는 기능들을 소개한다.

스프링 부트 애플리케이션 배포에서는 스프링 부트 로 개발된 웹 애플리케이션을 배포하는 방법을 살펴본다. 편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는 과정을 살펴 보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

빌드 도구 플러그인에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다. 스프링 부트 는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 빌드 플러그인을 제공하고 있다. 이 빌드 플러그인을 이용하여 빌드할 때 고려할 수 있는 다양한 선택사항들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '실행가능한' 배포본을 만들어내는 교묘한 기술 '재포장(리패키징, Repackage)' 에 대해 설명한다.

스프링 부트 활용 안내에서는 스프링 부트 로 개발하는 과정에서 고려해볼 다양한 확장 포인트들을 살펴본다. 스프링 부트 는 기본적으로 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트 를 사용하는 개발자에게 다양한

확장 포인트를 제공하고 있다. 이는 스프링 부트 의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

정리 은 앞에서 다뤘던 내용들을 정리한다. 미리 결론부터 이야기하자면 '자바 개발자+스프링 프레임워크 사용자' 라면 스프링 부트 를 능숙하게 쓰면 좋다는 것이다.

책을 따라하기 위해 필요항목

스프링 부트 의 시스템 요구사항 을 보면 1.4.0.RELEASE 은 Java 7 과 Spring Framework 4.3.2.RELEASE 혹은 상위버전을 요구한다. Java 6도 지원을 하지만 사용하는 컨테이너의 종류에 따라서 제한되거나 영향을 받을 수 있다. 책에서 작성한 코드들 중에 일부는 Java 8에서 제공하는 Optional, stream() 등을 사용하기에 Java 8을 설치하고 작성하길 바란다.

IDE 는 STS 와 IntelliJ 등이 있는데 이 책에서는 STS 를 기준으로 한다. 스프링 부트 에 대한 기능 지원이 괜찮기도 하고 이클립스 기반이라 많은 개발자들에게 익숙하고 무료로 사용할 수 있다는 강점(?)이 있어 부담없이 사용할 수 있다.

대상독자

스프링 부트 를 기반으로 한 개발에 관심을 가지고 이제 막 예제를 만들어보려 하거나 스프링 프레임워크에 대한 기본적인 지식이 있는 개발자를 대상으로 한다. 스프링 부트를 제대로 활용하기 위해서는 스프링 프레임워크와 그에 기반되는 지식을 가지고 있는 것이 좋다. 스프링 부트는 'spring + spring-boot application + auto-configuration + build-tool + 3rd party' 의 조합이기 때문에 자바 프로그래밍을 처음 시작하는 이들에게는 조금은 부담스러울 수 있다. 스프링 부트 를 제대로 사용하려면 스프링 에 대한 공부도 병행해야 한다.

관례(Convention)

책을 읽어가다보면 반복적으로 사용되는 표현들이 있을 것이다. 코드는 다음과 같이 작성된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight();
    }
}
```

코드 중에서 중요한 부분에 대해서는 다음과 같이 코드 오른쪽에 숫자가 매겨지고 하단에 그 숫자에 설명이 기술된다.

```
public class Honeymon {
    private String firstName;
    private String lastName;
    private Gender gender;
    private Integer age;
    private String site;
    private Integer weight;

    public Fat eat(Food food) {
        this.weight += food.getWeight(); ①
    }
}
```

① 먹은 만큼 찐다! 만고불변의 법칙!

중요한 단어나 문장에 대해서는 **굵은 글씨**로 처리한다. **스프링 부트**는 프레임워크다. 문장 중간에 노출되는 클래스명과 같은 코드성 표현에 대해서는 **Honeymon**으로 표현한다.

기억해두었으면 하는 부분들에 대해서는 다음과 같이 **노트**로 표기한다.

NOTE 노트

알아두면 유용한 것에 대해서는 다음과 같이 **유용함**으로 표기한다.

TIP 유용함!

인용한 것에 대해서는 **인용**으로 표기한다.

인용

용어나 개념을 소개하는 경우에는 한국어로 표현하는 것을 선호하는 편이다. 첫 표기시에는 **자동구성(Configuration)**과 **설정(SetUp)**과 같이 한국어와 영어를 병행표기하고 이후 표기에서는 한국어만 표기한다. 적절한 한국어 표현을 찾기 어려운 경우에는 **액추에이터(Actuator)**처럼 외래어 표기법을 따른다.

이 표기에 대해서는 의견이 분분하다. 어떤 이들은 영어를 그대로 표기하는 게 좋다하는 사람도 있지만, 나는 억지스러울지도 모르겠지만 한국어로 표기하고자 하는 노력을 기울이고 있다. 그렇다고 해서 나만 이해할 수 있는 엉터리 표기는 피하려고 노력한다. 이에 대해서는 각자의 방식으로 접근하고 가장 효율적인 방법을 선택하는 것이 적합하다고 본다.

예제

이 책에서 사용된 예제는 <https://github.com/ihoneymon/boot-spring-boot>에서 내려받을 수 있다. 메이븐이나 그레이들 없이 실행할 수 있도록 래퍼(wrapper)가 구성되어 있지만 이 래퍼를 실행하기 위해서는 JVM이 있어야 한다. 코드의 기본구성은 JDK8을 기준으로 작성되어 있다. 필요하다면 리파지토리를 복제(clone)해서 JDK7

에서도 실행가능하게 구성하고 공유해주면 다른 개발자들에게 도움이 될 것이다.

Chapter 1. 시작하기

스프링 부트 에 대해서 간단하게 소개하면서 스프링 부트 의 기본적인 개발흐름을 눈에 익혀본다. 스프링 부트는 그 이전에 스프링 기반의 웹 애플리케이션을 개발하던 때와는 확연하게 다른 개발경험을 개발자들에게 선사해 주었다. 이는 다른 언어가 가지고 있던 장점들을 자바 개발자들에게 제공하여 '빠른 프로토타이핑' 및 '개발/운영' 까지 이어지도록 해주는 근사한 경험이었다. 그 안에는 문맥(**Context**) 라고 하는 흐름이 구성(**Configuration**) 이라는 개념으로 변모해가는 역동적인 변화가 있었다.

Spring Boot makes it easy to create stand-alone, production-grade Spring based Applications that you can "just run". We take an opinionated view of the Spring platform and third-party libraries so you can get started with minimum fuss. Most Spring Boot applications need very little Spring configuration.

기능들:

- Create stand-alone Spring applications
- Embed Tomcat, Jetty or Undertow directly (no need to deploy WAR files)
- Provide opinionated 'starter' POMs to simplify your Maven configuration Automatically configure Spring whenever possible
- Provide production-ready features such as metrics, health checks and externalized configuration
- Absolutely no code generation and no requirement for XML configuration

1.1. 스프링 부트 소개

스프링 부트는 개발 플랫폼이다. 스프링 프레임워크를 기반으로 하여 관례적인 자동구성을 제공하고 자바 개발환경에서 사용되는 기능들을 쉽게 추가하고 사용할 수 있도록 한다.

스프링 부트 공식 사이트(<http://projects.spring.io/spring-boot>) 에 접속해보면 스프링 부트 이 제공하는 기능 정의를 통해 그 특징을 파악하는데 도움이 될 것이다.

- 단독으로 실행가능한 스프링 애플리케이션을 생성할 수 있다.
- WAR 파일로 배포하지 않고 내장 컨테이너로 톰캣, 제티 혹은 언더투를 선택하여 바로 실행할 수 있다.
- 'starter' POM 을 제공하여 메이븐 구성을 간단하게 해준다.
- 스프링 에 대한 자동구성을 제공한다.
- 출시가능한 수준의 측량, 건강점검과 외부 구성 등 운영에 필요한 다양한 기능을 제공한다.
- XML 구성을 위한 코드를 생성하거나 필요로 하지 않는다.

위의 여섯가지 기능으로 스프링 부트 란 어떤 녀석인지 짐작해볼 수 있을 것이다. 이 기능들에 대해서는 차차 살펴보기로 하자. 가장 큰 변화라고 할 수 있는 부분 중에 하나는 더이상 **XML 구성**을 필요로 하지 않는다는 것이다. 자바구성 (**JavaConfig**) 을 통해서 애플리케이션에 필요한 컴포넌트 스캔(**ComponentScan**) 과 스프링 빈(**Bean**) 등록 등을 할 수 있게 되었다. 이마저도 관례적인 자동구성(**Auto-Configuration**)을 제공하여 설정에 대한 부담감을 줄여준다. 스프링 부트를 기반으로 개발하는 개발자는 환경구성에 대한 부담없이 비즈니스 로직구현에만 집중할 수 있도록 해준다.

이런 기능을 제공하는 스프링 부트는 **java -jar** 명령으로 실행할 수 있는 실행가능한(executable) jar 로 빌드하고

배포할 수 있는 특징을 가진다. 다양한 환경에서 실행 운영할 수 있는 특징은 최근에 많은 관심을 받고 있는 MSA(MicroService Architecture)를 지원하는 강점으로 부각된다. 이외에도 스프링 부트가 가지고 있는 특징과 제공하는 편의 기능들에 대해서 찬찬히 살펴보도록 하자.

1.2. 시스템 요구사항

스프링 부트는 기본적으로 [Java 7](#) 과 Spring Framework 4.3.2.RELEASE 혹은 상위버전을 요구한다. 스프링 부트 버전에 따라 적재된 스프링 프레임워크의 버전은 다르니 이를 확인하기 바란다. 스프링 부트는 정식버전을 출시하면서 스프링 4.0.X 버전을 적재하고 있기에 이에 맞춰 애플리케이션의 기능을 설계하고 구현하기를 권한다. 필요하다면 Java 버전과 스프링 버전을 변경할 수 있지만 그에 따라 별도로 구성을 추가하여야할 것들이 있고 최근에 나온 플러그인들이나 라이브러리들이 Java 7에 맞춰진 경우가 많아서 특별한 경우가 아니라면 Java 7을 기준으로 사용하길 권한다.

NOTE

스프링 부트 개발팀에서는 가능하면 Java 8 을 사용하기를 권한다. 나 역시, Java 8 이 제공하는 의 혜택을 볼 수 있도록 하길 Java 8 을 사용하기를 권한다.

스프링 부트 와 관련된 구성과 의존성을 확보하기 위해서는 메이븐 혹은 그레이들 빌드도구가 필요하다. 이 빌드 도구를 통해서 메이븐 저장소에서 starter POM 을 가져오고 그와 관련된 라이브러리들을 추가할 수 있기 때문이다. 이 책에서는 그레이들 을 중심으로 이야기를 진행한다. 메이븐 혹은 그레이들 래퍼(wrapper) 를 이용하게 되면 다른 팀원들은 프로젝트 빌드를 위해서 메이븐 혹은 그레이들 을 설치하지 않아도 된다. 이에 대한 설명은 [빌드 시스템](#) 에서 자세히 할 예정이다.

Java 사용버전을 고려하는 데 있어서 가장 큰 영향을 주는 것은 사용하는 컨테이너가 무엇이나 하는 것이다. 스프링 부트는 기본적으로 톰캣(tomcat)을 내장형 컨테이너로 사용하지만 필요에 따라 손쉽게 제티(jetty)와 언더투우(Undertow) 로 변경이 용이하다.

NOTE

웹소켓 등 컨테이너에 따라 의존성을 가지고 있는 라이브러리 등이 영향을 받기 때문에 컨테이너를 선정할 때는 이런 영향도를 고려하기 바란다. 간단한 예로 웹소켓([JSR-356](#)) 구현체가 각기 다르기에 컨테이너 변경시 영향을 많이 받을 수 있다.

- [Tomcat WebSocket\(org.apache.tomcat.websocket\)](#)
- [Jetty WebSocket\('org.eclipse.jetty.websocket'\)](#)

1.3. 스프링 부트 설치

1.3.1. 설치방법 소개

스프링 부트 CLI 를 사용한 방법은 생략한다. 내가 많이 안쓴다.

메이븐 설치

그레이들 설치

1.4. 스프링 부트 애플리케이션 개발

1.4.1. 프로젝트 생성

1.4.2. 의존성 추가

1.4.3. 코드 작성하기

[코드 구조](#)에서 설명하긴 하겠지만, 애플리케이션 메인 클래스를 제외한 새롭게 생성하는 클래스들은 기본 `default` 패키지에 두지 않기를 권한다.

1.4.4. 실행하기

보다 자세한 내용은 [애플리케이션 실행하기](#)에서

1.4.5. 실행가능한 `jar` 생성

1.5. 이어질 이야기

Chapter 2. 첫 스프링 부트 애플리케이션 살펴보기

기능을 구현하는데 필요한 다른 라이브러리들을 손쉽게 추가할 수 있는 '의존성 관리', 추가된 '의존성' 라이브러리들을 바로 사용할 수 있도록 스프링 부트 개발팀이 권장하는 '자동구성(Auto-Configuration)', 개발 생산성을 높여주고 확장성을 부여하는 '스프링 프레임워크'와의 조합, 내장컨테이너를 이용한 빠른 실행과 재가동 시간을 줄이는 '개발자 도구' 등 스프링 부트 를 이용하는데 알아둬야할 것들을 살펴본다.

2.1. 빌드 시스템

2.1.1. 의존성 관리

2.1.2. 메이븐

2.1.3. 그레이들

2.1.4. 스타터

2.2. 코드 구조

2.2.1. "기본" 패키지 이용

2.2.2. 애플리케이션 메인 클래스 위치

2.3. 구성(Configuration) 클래스

2.3.1. 추가적인 구성클래스 불러오기

2.3.2. XML 구성 불러오기

2.4. 자동구성(Auto-configuration)

스프링 부트 개발팀에서 범용적으로 사용되는 구성을 JavaConfig 와 ~.properties 를 구성하여 사용하려는 기능의 라이브러리가 추가되면 조건에 따라 자동적으로 선택되는 구성클래스에 대한 이야기

2.4.1. 자동구성 대체하기

미리 마련되어 있는 인터페이스와 추상 클래스를 이용하여 이용자가 자신에게 맞춰 재정의하여 사용한다.

2.4.2. 특정 자동구성 비활성화하기

사전 계획에 따라 의존성은 추가해줬지만 당장은 필요없는 경우에 자동구성을 꺼둔다.

2.5. 스프링 빈과 의존성 주입

2.6. @SpringBootApplication 애노테이션 이용

2.7. 애플리케이션 실행하기

2.7.1. IDE 에서 실행하기

2.7.2. 패키징된 애플리케이션 실행하기

2.7.3. 메이븐 플러그인으로 실행하기

2.7.4. 그레이들 플러그인으로 실행하기

2.7.5. 핫 스와핑

2.8. 스프링 부트 개발자도구

2.8.1. 기본 속성들

2.8.2. 자동 재시작

2.8.3. 전역 설정

2.8.4. 원격 애플리케이션

2.9. 출시를 위한 애플리케이션 패키징

2.10. 이어질 이야기

Chapter 3. 스프링 부트 기능

스프링 부트의 진입점(Entry point)가 되는 `SpringApplication`을 시작으로 해서 배포를 위해 구성된 배포파일을 다시 빌드하지 않아도 실행환경에 따라 설정값을 변경할 수 있으며, 프로파일을 통해서 '로컬','단위테스트','개발','운영' 등의 독립적인 실행환경별로 구성하고, 다양한 생태계와 연동하고, 필요에 따라 추가적인 구성을 작성하는 과정 등을 살펴보게 된다. 이 장을 읽으면서 스프링 부트의 매력에 흠뻑 취하게 될 것이다.

3.1. 스프링애플리케이션(SpringApplication)

3.1.1. 배너 재정의

3.1.2. `SpringApplication` 재정의

3.1.3. 유연한(`Fuent`) 빌더 API

3.1.4. 애플리케이션 이벤트 및 리스너

3.1.5. 웹 환경

3.1.6. 애플리케이션 실행 인자에 접근

3.1.7. `ApplicationRunner` 혹은 `CommandLineRunner` 사용하기

3.1.8. 애플리케이션 종료

3.1.9. 관리자 기능

3.2. 외부 구성

3.2.1. 랜덤값 구성

3.2.2. 명령줄 속성 접근

3.2.3. 애플리케이션 속성 파일

3.2.4. 프로파일 정의 속성

3.2.5. 속성 내 치환자(placeholder)

3.2.6. `properties` 를 대신하여 `YAML` 사용하기

YAML 적재

{spring-environment} 에서 `properties` 처럼 `YAML` 사용하기

YAML 문서로 다중 프로파일 다루기

YAML 단점

YAML 목록 병합(Merging)

3.2.7. 보장형 구성 속성

서드파티 구성

느슨한 연결

`Properties` 관례

`@ConfigurationProperties` 유효성검사

`@ConfigurationProperties` 대 `@Value`

3.3. 프로파일

3.3.1. 활성화할 프로파일 추가하기

3.3.2. 프로파일을 프로그래밍적으로 설정

3.3.3. 프로파일 정의 구성 파일

3.4. 로깅

3.4.1. 로그 형식

3.4.2. 콘솔 출력

3.4.3. 파일 출력

3.4.4. 로그 레벨

3.4.5. 로그 구성 재정의

3.4.6. 로그백 확장

3.5. 웹애플리케이션 개발

3.5.1. 'Spring Web MVC framework'

3.5.2. 내장 서블릿 컨테이너 지원

서블릿, 필터 와 리스너

서블릿 컨텍스트 초기화

`EmbeddedWebApplicationContext`

내장 서블릿 컨테이너 재정의

JSP 제한

3.6. 보안

3.6.1. OAuth2

공인(`Authorization`) 서버

자원(`Resource`) 서버

3.6.2. 사용자 정보 속 토큰 유형

3.6.3. 사용자정보 `RestTemplate` 재정의

클라이언트

`Single Sign On`

3.6.4. 액츄에이터 보안(`Actuator Security`)

3.7. SQL 데이터베이스 이용

3.7.1. `DataSource` 구성

3.7.2. `JdbcTemplate` 사용

3.7.3. JPA 그리고 '스프링 Data'

3.7.4. H2 사용

프로파일 구성

H2 웹콘솔 사용

3.8. NoSQL 기술 이용

3.8.1. 레디스(Redis)

3.8.2. 몽고DB(MongoDB)

몽고DB 데이터베이스 연결

MongoTemplate

스프링 Data 몽고DB 리파지토리

내장 몽고

3.8.3. 솔(Solr)

솔 연결하기

스프링 Data 솔 리파지토리

3.8.4. 엘라스틱서치(Elasticsearch)

스프링 Data 를 통해 엘라스틱서치 연결

스프링 Data 엘라스틱서치

3.9. 캐시

3.9.1. 지원하는 캐시 제공자(Provider)

Generic

JCache

EhCache 2.x

Hazelcast

Redis

Guava

Simple

None

3.10. REST 서비스 호출

3.10.1. `RestTemplate` 재정의

3.11. 스프링 세션

3.12. 테스트

3.12.1. 테스트 관련 의존성

3.12.2. 스프링 애플리케이션 테스트

3.12.3. 스프링 부트 애플리케이션 테스트

테스트 구성 탐색

테스트 구성 제외

랜덤포트 이용

빈에 대한 목(Mock) 과 스파이(Spy) 처리

자동구성된 테스트

자동구성된 JSON 테스트

자동구성된 스프링 MVC 테스트

자동구성된 Data JPA 테스트

자동구성된 REST 클라이언트

자동구성된 스프링 REST Docs 테스트

스폭(Spock) 을 사용하여 스프링 부트 애플리케이션 테스트

3.12.4. 테스트 유틸리티

`ConfigFileApplicationContextInitializer`

`EnvironmentTestUtils`

`OutputCapture`

3.13. 웹소켓(WebSocket)

3.14. 웹 서비스(Web Service)

3.15. 자동구성 직접 생성하기

3.15.1. 자동구성 빈 이해하기

3.15.2. 자동구성 대상자 위치

3.15.3. 조건 애너테이션

클래스 조건

빈 조건

속성 조건

자원 조건

웹애플리케이션 조건

SpEL 표현식 조건

3.15.4. 스타터 직접 생성하기

이름짓기

자동구성 모듈

스타터 모듈

3.16. 이어질 이야기

Chapter 4. 스프링 부트 액츄에이터: 출시준비기능

개발하여 배포하고 운영되고 있는 스프링 부트 를 모니터링하고 그 정보를 수집하고 경우에 따라 쉽게 제어할 수 있는 기능들을 소개한다.

4.1. 출시준비 기능 활성화

4.1.1. 종단점(Endpoint)

4.1.2. 종단점 재정의

4.1.3. 액츄에이터 MVC 종단점 를 위한 하이퍼미디어

4.1.4. CORS 지원

4.1.5. 재정의 종단점 추가하기

4.1.6. 건강정보

4.1.7. `HealthIndicator` 를 사용한 보안

4.1.8. 애플리케이션 정보

자동구성된 `InfoContributors`

재정의한 애플리케이션 정보

깃(Git) 커밋 정보

빌드(build) 정보

`InfoContributors` 재정의 작성

4.2. HTTP 로 모니터링하고 관리하기

4.2.1. 종단점 에 대한 노출강도 보안관리

4.2.2. 관리 종단점 경로 재정의

4.2.3. 관리 서버 포트 재정의

4.2.4. 관리를 위한 SSL 재정의

4.2.5. 관리 서버 주소 재정의

4.2.6. HTTP 종단점 비활성화

4.2.7. HTTP 건강 종단점 접근 제한

4.3. 원격셀을 통해 모니터링하고 관리하기

4.3.1. 원격셀로 접속

4.3.2. 원격셀 확장

4.3.3. 원격셀 플러그인

4.4. 측량(Metrics)

4.4.1. 시스템 측량

4.4.2. 데이터소스 측량

4.4.3. 캐시 측량

4.4.4. 톰캣 세션 측량

4.4.5. 측량 기록

4.4.6. 공개 측량 추가

4.5. 감시

4.6. 추적

4.7. 프로세스 모니터링

4.7.1. 확장 구성

4.7.2. 프로그래밍적인 처리

4.8. 이어질 이야기

Chapter 5. 스프링 부트 애플리케이션 배포

편리하고 막강한 기능을 제공하는 클라우드 서비스들이 많이 등장했다. 그 클라우드 서비스에 우리가 만든 앱을 배포하는 과정을 살펴보면서 자신에게 맞는 클라우드 서비스를 선택하고 관리해보자.

5.1. 클라우드 배포

5.1.1. 아마존웹서비스(AWS, Amazon Web Service)

5.1.2. 구글앱엔진

5.1.3. 클라우드 파운드리(Cloud Foundry)

5.2. 스프링 부트 애플리케이션 설치

5.2.1. 유닉스/리눅스 서비스

5.3. 마이크로소프트 윈도우즈 서비스

5.4. 이어질 이야기

Chapter 6. 빌드 도구 플러그인

에서는 '실행가능한(Executable)' 배포본(jar or war)를 만들어내는 '빌드 도구' 를 살펴보도록 한다. 스프링 부트는 자바 개발시 주로 사용하는 빌드도구인 메이븐과 그레이들 에서 사용할 수 있는 빌드 플러그인을 제공하고 있다. 이 빌드 플러그인을 이용하여 빌드할 때 고려할 수 있는 다양한 선택사항들을 제공하여 개발자의 다양한 욕구를 수용하고 있다. 여기서 '실행가능한' 배포본을 만들어내는 교묘한 기술 '재포장(리패키징, Repackage)' 에 대해 설명한다.

6.1. 스프링 부트 메이븐 플러그인

6.1.1. 플러그인 포함

6.1.2. 실행가능한 jar 와 war 파일 패키징

6.2. 스프링 부트 그레이들 플러그인

6.2.1. 플러그인 포함

6.2.2. 그레이들 의존성 관리

6.2.3. 실행가능한 jar 와 war 파일 생성

6.2.4. 스프링 부트 플러그인 구성

6.2.5. 리패키징 구성

6.2.6. 그레이들 플러그인이 어떻게 동작하는지 이해하기

6.2.7. 그레이들 을 이용해서 메이븐 리파지토리에 아티팩트 배포

6.3. 다른 빌드 시스템 지원

6.3.1. 리패키징 아카이브

6.3.2. 내포된 라이브러리

6.3.3. 메인 클래스 찾기

6.3.4. 리패키지 구현 예제

6.4. 이어질 이야기

Chapter 7. 스프링 부트 활용 안내

스프링 부트는 기본적으로 개발팀에서 권장(Recommend) 하는 방식이 있다. 그렇지만 스프링 부트를 사용하는 개발자에게 다양한 확장 포인트를 제공하고 있다. 이는 스프링 부트의 근간이 되는 스프링 프레임워크가 가지고 있는 특징이기도 하다. '다 써볼 수 있을까?' 싶을만큼 너무나 다양하고 방대한 내용을 다루고 있다.

7.1. 스프링 부트 애플리케이션

7.1.1. 자동구성 문제해결

7.1.2. `Environment` 혹은 `ApplicationContext` 가 시작하기 전에 재정의하기

7.1.3. `ApplicationContext` 계층 조직(부모 혹은 루트 컨텍스트 추가)

7.1.4. 비-웹애플리케이션 생성

7.2. 속성 및 구성

7.2.1. 빌드 시 자동으로 속성 확장

7.2.2. `SpringApplication` 외부 구성

7.2.3. 애플리케이션 속성 외장파일 위치변경

7.2.4. 명령줄 인자 축약사용

7.2.5. 외부속성에 대한 YAML 사용

7.2.6. 스프링 프로파일 활성화

7.2.7. 환경 의존적인 구성 변경

7.2.8. 외부속성에 대한 빌트인 옵션(built-in option) 발견

7.3. 내장서블릿 컨테이너

7.3.1. 애플리케이션에 서블릿, 필터 혹은 리스너 추가하기

7.3.2. HTTP 포트 변경

7.3.3. 할당되지 않은 HTTP 포트 무작위 사용

7.3.4. 실행시 HTTP 포트 발견

7.3.5. SSL 구성

7.3.6. 접근로그 구성

7.3.7. 프론트엔드 프록시 서버 이용

7.3.8. 톰캣 구성

7.3.9. 톰캣을 통한 다중 커넥터 활성화

7.3.10. 톰캣 7.x 혹은 8.0 이용

7.3.11. `@ServerEndpoint` 를 사용한 웹소켓 종단점 생성

7.3.12. HTTP 응답 압축 활성화

7.4. 스프링 MVC

7.4.1. JSON REST 서비스 작성

7.4.2. XML REST 서비스 작성

7.4.3. 잭슨(Jackson) 오브젝트매퍼(`ObjectMapper`) 재정의

7.4.4. `@ResponseBody` 렌더링 재정의

7.4.5. Multipart 파일 업로드 제어

7.4.6. 스프링 MVC `DispatcherServlet` 전환

7.4.7. 기본 MVC 구성 전환

7.4.8. `ViewResolver` 재정의

7.5. HTTP 클라이언트

7.5.1. `RestTemplate` 가 프록시를 사용하도록 구성

7.6. 로깅

7.6.1. Logback 구성

7.6.2. Log4j 구성

7.7. 데이터 접근

7.7.1. DataSource 구성

7.7.2. DataSource 복수 정의

7.7.3. 스프링 Data 리파지토리 이용

7.7.4. 스프링 구성으로부터 @Entity 정의 분리

7.7.5. JPA 속성 구성

7.7.6. EntityManagerFactory 재정의 이용

7.7.7. 복수 EntityManager 이용

7.7.8. 전통적인 persistence.xml 이용

7.7.9. 스프링 Data JPA 와 몽고 리파지토리 이용

7.7.10. REST 종단점 로 스프링 Data 리파지토리 노출

7.8. 데이터베이스 초기화

7.8.1. JPA 를 이용한 데이터베이스 초기화

7.8.2. Hibernate 를 이용한 데이터베이스 초기화

7.8.3. 스프링 JDBC 를 이용한 데이터베이스 초기화

7.8.4. 스프링 배치 를 이용한 데이터베이스 초기화

7.8.5. 고차원 데이터베이스 마이그레이션 도구 사용

Flyway

Liquibase

7.9. 배치 애플리케이션

7.9.1. 구동시 스프링 배치 작업 실행

7.10. 액츄에이터

7.10.1. 액츄에이터 종단점 의 포트 혹은 주소 변경

7.10.2. `whitelabel` 오류 페이지 재정의

7.10.3. 액츄에이터 과 Jersey

7.11. 보안

7.11.1. 스프링 부트 보안 구성 전환

7.11.2. `AuthenticationManager` 변경하고 사용자 계정 추가

7.11.3. 프록시 서버 뒤에서 실행될 때 HTTPS 활성화

7.12. 핫스와핑(Hot swapping)

7.12.1. 정적 콘텐츠 재적재

7.12.2. 컨테이너 재시작없이 템플릿 재적재

7.12.3. 빠른 애플리케이션 재시작

7.13. 빌드

7.13.1. 빌드 정보 생성

7.13.2. 깃 정보 생성

7.13.3. 의존성 버전 재정의

7.13.4. Java 6 에서 실행하기

7.14. 전통적 배포

7.14.1. 배포가능한 war 파일 생성

7.14.2. 오래된 서블릿 컨테이너에 배포가능한 war 파일 생성

7.14.3. 기존 애플리케이션을 스프링 부트 로 변환

7.14.4. 오래된 컨테이너(Servlet 2.5)에 war 배포

Chapter 8. 정리

'자바 개발자+스프링 프레임워크 사용자' 라면 **스프링 부트** 를 능숙하게 쓰면 좋다는 것이다.

부록

Appendix A: 개발환경 설정

JDK설치

STS 설치

그레이들 설치

Appendix B: 개발 시 참고사항

스프링부트 레퍼런스 문서 참고

관련 자동구성 클래스 확인

`debug=true` 를 이용한 활성화된 자동설정과 비활성화된 자동설정 확인

실행가능한 jar 구조

공통 애플리케이션 속성들

스택오버플로에서 스프링부트 관련 질문 찾기

Appendix C: 참고문헌

Spring Framework

Spring Boot

Spring Security

Spring Test